COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/20/2026	**DATE SUBMITTED:**	10/20/2026
NAME:	Jose Martin C. Rodulfa Dominic Rodnee A. Panti	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14 **Algorithm Complexity**

PROCEDURE OUTPUT

	Best Case	**Worst Case**	**Analysis**
Merge Sort	O(nlogn)	O(nlogn)	Merge sort will have a consistent best & worst case scenario of O(nlogn) due to the fact that the "dividing" of the array/data structure will ALWAYS happen.
Quick Sort	O(nlogn)	O(n^2)	Quick Sort will have a varying best and worst case, with the best case happening if the pivot consistently splits the array in halves, while the worst case happening if the pivot splits the array unequally, rendering it more tedious to compare elements.

Table 14-1. Best and Worst Case Analysis using Theoretical Tools

	Merge Sort	**Quick Sort**

Best Case	100:
	`Shell Sort Time: 4.8 microseconds`
	1000:
	`Merge Sort Time: 6.5 microseconds`
	10,000:
	`Merge Sort Time: 789.9 microseconds`
		100:
		`Quick Sort Time: 4.4 microseconds`
		1000:
		`Quick Sort Time: 4.1 microseconds`
		10,000:
		`Quick Sort Time: 724.4 microseconds`

Worst Case	100:
	`Merge Sort Time: 6.5 microseconds`
	100:
	`Merge Sort Worst-Case Execution Time: 74.7 microseconds`
	10,000:
	`Merge Sort Worst-Case Execution Time: 871.9 microseconds`
		100:
		`Quick Sort Time: 4.7 microseconds`
		1000:
		`Quick Sort Worst-Case Execution Time: 1014.2 microseconds`
		10,000:
		`Quick Sort Worst-Case Execution Time: 96272.9 microseconds`

Table 14-2. Best and Worst Case Analysis using Experimental Tools

COMPUTER ENGINEERING

QUEZON CITY

ANSWERS TO SUPPLEMENTAL ACTIVITY

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	`Binary Search Time: 14.7 microseconds`	`Linear Search Time: 11.6 microseconds`
10,000	`Binary Search Avg Time : 20.3872 microseconds`	`Linear Search Avg Time : 24.8414 microseconds`
100,000	`Binary Search Time: 39.3us microseconds`	`Linear Search Time: 55.3 microseconds`
Space Complexity	O(logN)	O(N)

ANALYSIS:

- Both searching algorithms experienced searching for a non-existent number in a ***randomly generated array*** of varying sizes, namely 1,000, 10,000, and 100,000 elements in the array. We can observe there that the theoretical worst-case time complexity of both searching algorithms are partially true, observing the considerable gap in processing time of both as the array size increases. In terms of space complexity, Linear Search has O(N) space complexity due to the structure of the function being a linear for loop, while the Binary Search has O(logN) space complexity due to the nested for loop and separated std::cout line.

CONCLUSION

- In this HOA, I was able to further utilize my knowledge in theoretical time and space complexity analysis with the past functions that we've used in the course. I would say that I am decently experienced in assessing the time and space complexity of a given function, as well as conducting the experimental test for the time complexity of a said function by utilizing the <chrono> library. For the procedure, it was a relatively straightforward task where we compared the best and worst case experimental time complexity of our created merge sort and quick sort functions. This is where we explored how input size and background activity is the catalyst for varying runtimes. This knowledge was carried over when we did the supplementary activity, where we tested the runtime of both searching algorithms with an exponentially increasing array size using randomly generated integers. We observed the gaps in runtime as the input size got bigger, correctly defining the best and worst case scenarios of searching algorithms in C++.

Rodulfa

- At this HOA I was able to apply my knowledge of theoretical time and space complexity analysis together with C++ multithreading techniques using the <thread> library. I can say that I have a fairly good level of experience in assessing the efficiency of algorithms, in handling shared variables, and in implementing thread synchronization barriers with .join(). The procedure itself was rather a simple one involving an observation of how the manipulation of global variables behaves when multiple threads are involved and a examination of how the order in which threads are executed affects the final output. It was in this part that we looked at how concurrent execution and thread scheduling cause variations in runtimes and in the sequence of outputs. This understanding was then applied when we studied parallel algorithms such as multithreaded Merge Sort, in which dividing the tasks among concurrent threads greatly improves sorting performance as compared to single-threaded versions. In general, I believe that I did a good job in carrying out the multithreaded coding and in interpreting the output from the threads. As for the future, my main area for improvement is learning how to use synchronization primitives such as mutexes (std::mutex) in order to avoid race conditions without having to depend solely on sequential thread joining.

Panti

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.